L1 · Pasul 3 — Backend-ul în compose

Flavius · one-user-management-api pornit lângă mysql, din imaginea ta de pe Docker Hub

La final, un singur `docker compose up -d` pornește baza și API-ul, iar Swagger-ul îl deschizi în browser pe `localhost:8111`. Același terminal: VS Code, profilul **WSL: Ubuntu**, Docker Desktop pornit. **CONTUL-TAU** = același cont de Docker Hub ca la pasul 2.

1. Al doilea serviciu în docker-compose.yaml

Te întorci în rădăcina repo-ului (la pasul 2 ai rămas în `proiecte/one-user-management-java`):

```
cd ~/devops-initiere
```

Deschizi `docker-compose.yaml` (cel de la pasul 1) și îl faci exact așa; nou e doar blocul `one-user-management-api`, la aceeași indentare cu `mysql:`. Rândul `SPRING_DATASOURCE_URL` rămâne un singur rând, fără spații.

```
services:
  mysql:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: root
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-proot"]
      interval: 5s
      timeout: 5s
      retries: 12
      start_period: 30s
    restart: unless-stopped

  one-user-management-api:
    image: CONTUL-TAU/one-user-management-api:1.0
    ports:
      - "8111:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/user_management_db?createDatabaseIfNotExist=true
    depends_on:
      mysql:
        condition: service_healthy
    restart: unless-stopped

volumes:
  mysql_data:
```

```
docker compose config --quiet
```

Trebuie să vezi: nimic. O eroare cu `yaml` = indentare greșită.

2. Pornește tot

```
docker compose up -d
```

Trebuie să vezi: `Container devops-initiere-mysql-1 Healthy`, abia apoi `Container devops-initiere-one-user-management-api-1 Started`: API-ul așteaptă ca baza să fie gata.

Capcană: `pull access denied for …/one-user-management-api, repository does not exist` → numele din `image:` nu e exact cel urcat la pasul 2; îl corectezi și rulezi din nou `docker compose up -d`.

```
docker compose ps
```

Trebuie să vezi: două rânduri: `mysql-1` cu `Up … (healthy)` și `one-user-management-api-1` cu `Up …` și `0.0.0.0:8111->8080/tcp`.

3. Log-ul API-ului

```
docker compose logs one-user-management-api
```

Trebuie să vezi: spre final `Tomcat started on port 8080 (http)` și `Started UserManagementApplication in … seconds`. Dacă nu apare încă, aștepți câteva secunde și repeți comanda.

4. Swagger în browser

În browser-ul din Windows deschizi `http://localhost:8111/swagger-ui.html`. **Trebuie să vezi:** pagina Swagger cu secțiunea `Utilizatori`. La `GET /api/users` → **Try it out** → **Execute**: cod `200` și lista cu cei 5 useri puși la prima pornire.

Temă: online-shop-api, al treilea serviciu

Refaci pașii 2 și 3 pentru `proiecte/online-shop-api-java`. Diferențe: proiectul e pe Java 17, deci în `Dockerfile` folosești `maven:3.9-eclipse-temurin-17`, `eclipse-temurin:17-jre` și `EXPOSE 8082`; imaginea se numește `online-shop-api:1.0`, apoi `CONTUL-TAU/online-shop-api:1.0`. În compose, sub `one-user-management-api`:

```
  online-shop-api:
    image: CONTUL-TAU/online-shop-api:1.0
    ports:
      - "8104:8082"
    environment:
      DB_URL: jdbc:mysql://mysql:3306/online_shop_db?createDatabaseIfNotExist=true
    depends_on:
      mysql:
        condition: service_healthy
    restart: unless-stopped
```

Trebuie să vezi: trei rânduri în `docker compose ps`, `Started OnlineShopSpringApplication` în `docker compose logs online-shop-api` și Swagger-ul pe `http://localhost:8104/swagger-ui.html`.

Gata când…

`docker compose ps` arată API-ul `Up` lângă `mysql (healthy)`, log-ul are `Started UserManagementApplication` și `GET /api/users` din Swagger întoarce `200`.

În `notite-L1.txt` (rădăcina repo-ului) adaugi un rând: *Pasul 3: în compose API-ul găsește baza după numele serviciului (mysql:3306), iar depends_on cu service_healthy îl pornește abia când baza e gata.*